



Groupe IPIRNET

Institut Privé d'informatique, Réseau et Nouvelles

Etudes de Télécommunication

Autorisé par l'Etat sous N° : 3 /03/2/2003 Du :19/02/2003

Accrédité par l'Etat sous N°

21/DFP/F0301/199 du 29/11/2021



Filière Technicienne Spécialisée en Développement d'Informatique (TSDI)

Rapport de Projet

Sous le thème

Développement d'une application de gestion pour le centre de formation IPIRNET en utilisant Python et l'algorithmique

Titre de projet :

DÉVELOPPEMENT D'UNE APPLICATION DE GESTION DES STAGIAIRES DU CENTRE IPIRNET EN UTILISANT PYTHON ET LES CONCEPTS DE L'ALGORITHMIQUE

Réalisé par :

ZINEB FRINDY : Etudiante en 1^{ère} année Filière Technicienne Spécialisée en
Développement d'Informatique

Sous la direction de :

Pr. ABDOUSSI : Encadrant Académique

Année de Formation : 2024 – 2025

Dédicace

À ceux qui ont allumé en moi la flamme de la persévérance,

À **ma famille bien-aimée**, pour son amour inconditionnel, son soutien moral et sa foi en moi à chaque étape.

À **Monsieur Abdoussi**, mon enseignant de programmation, pour sa disponibilité, sa rigueur, et l'inspiration qu'il m'a transmise à travers ses cours.

À **l'établissement IPIRNET**, un lieu de savoir, d'opportunités et d'épanouissement.

À tous ceux et celles qui, de près ou de loin, ont contribué à la réalisation de ce projet,

Je vous dédie humblement ce travail, avec tout mon respect et ma reconnaissance.

Remerciements

Ce projet représente bien plus qu'un simple exercice académique : il est le fruit d'un parcours d'apprentissage, de collaboration et de persévérance. À travers ces lignes, je souhaite exprimer ma gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je tiens tout d'abord à remercier très sincèrement **Monsieur Abdoussi**, mon professeur de programmation, pour la qualité de son encadrement, sa disponibilité, ainsi que pour ses conseils avisés qui ont su orienter mes efforts dans la bonne direction. Son expertise et sa pédagogie ont été des piliers essentiels tout au long de ce projet.

Je tiens également à exprimer ma reconnaissance envers le **Groupe IPIRNET**, qui m'a offert un environnement d'apprentissage stimulant, structuré et propice à l'épanouissement personnel et professionnel. Grâce à l'encadrement rigoureux et à l'approche pédagogique adoptée par l'établissement, j'ai pu développer des compétences solides et concrètes dans le domaine du développement informatique.

Je ne saurais oublier ma **famille**, sans qui rien n'aurait été possible. Leur **soutien indéfectible**, tant sur le plan moral que matériel, m'a permis de garder confiance en moi, même dans les moments de doute. Leur présence constante, leur patience et leurs encouragements ont été ma plus grande force.

Enfin, je remercie toutes les personnes qui m'ont encouragée, soutenue ou inspirée tout au long de ce projet. Vos mots, vos gestes et votre présence m'ont été précieux.

À chacun d'entre vous, j'adresse mes remerciements les plus sincères et les plus respectueux.

Table des matières

Dédicace	2
Remerciements	3
Introduction	5
1.1 Contexte du projet :	5
1.2 Objectifs du projet :	5
1.3 Méthodologie :	6
Cahier des charges	7
2.1 Présentation de l'Institut IPIRNET :	7
2.2 Analyse des besoins fonctionnels :	7
2.3 Analyse des besoins techniques :	7
2.4 Contraintes et spécifications :	8
Étude et conception	9
3.1 Modélisation des données (Tableaux, structures) :	9
3.2 Diagrammes fonctionnels :	9
3.3 Architecture de l'application :	10
3.4 Choix technologiques :	10
Développement	11
4.1 Présentation de l'environnement de développement :	11
4.2 Description des modules et fonctions :	11
4.3 Gestion des stagiaires :	12
4.4 Gestion des notes :	12
4.5 Validation et règles métier :	12
Tests et validation	13
5.1 Stratégie de test :	13
5.2 Tests unitaires et résultats :	13
5.3 Tests d'intégration :	14
5.4 Retours d'utilisation :	14
Difficultés rencontrées et solutions apportées	15
6.1 Manipulation des tableaux et structures :	15
6.2 Saisie et validation des données :	15
6.3 Gestion de la moyenne générale et des règles de validation :	16

6.4 Interface utilisateur en mode texte :	16
6.5 Organisation du projet et respect des consignes :	16
Conclusion et perspectives	17
Perspectives :	17
Annexes	18
8.1 Codes sources :	18
8.2 CAPTURES D'ÉCRAN DES MENUS :	22
8.3 DOCUMENTATION COMPLÉMENTAIRE	26
➤ Algorithme général de l'application	26

Introduction

1.1 Contexte du projet :

Dans le cadre de notre formation en **1ère année Technicien Spécialisé en Développement Informatique** au sein du **Groupe IPIRNET**, un projet pratique nous a été confié dans la matière **Programmation structurée (Python)**. Ce projet s'inscrit dans une approche pédagogique qui vise à appliquer de manière concrète les connaissances acquises en cours, notamment en **algorithmique** et en **programmation**.

Le projet proposé consiste à développer une **application de gestion des stagiaires** du centre de formation. Ce système permet de centraliser les données relatives aux apprenants et de faciliter leur traitement, à travers un ensemble de fonctionnalités automatisées : chargement des données, affichage des états, édition des bulletins de notes, validation des résultats, etc.

1.2 Objectifs du projet :

L'objectif principal de ce projet est de **concevoir et développer une application en Python**, fondée sur les **concepts de l'algorithmique**, pour la **gestion complète des stagiaires** du centre. Les objectifs pédagogiques et techniques sont les suivants :

- Approfondir la maîtrise du langage Python.
- Appliquer les notions de **programmation structurée** : procédures, fonctions, boucles, structures conditionnelles, etc.
- Utiliser des **structures de données**, principalement les tableaux (listes), pour gérer les informations.
- Concevoir une **interface en ligne de commande** claire à l'aide d'un **menu de choix** pour la navigation.
- Implémenter des fonctionnalités pratiques telles que :
 - L'ajout et la suppression de stagiaires,
 - La recherche et la modification des données,
 - L'affichage des bulletins de notes et de l'état par filière,
 - La validation automatique des résultats selon des règles définies.

1.3 Méthodologie :

Pour mener à bien ce projet, nous avons adopté une méthodologie structurée en plusieurs étapes :

- **Analyse du besoin** : étude du cahier des charges fourni par l'enseignant, identification des données à gérer, des traitements à appliquer et des résultats attendus.
- **Conception algorithmique** : élaboration des algorithmes de chaque fonction (ajout, recherche, calculs...), mise en place de la logique de traitement.
- **Développement en Python** : traduction des algorithmes en code Python, utilisation des tableaux, gestion des entrées/sorties, organisation du code par procédures.
- **Tests et validation** : exécution du programme avec des cas réels, corrections des erreurs, amélioration des résultats.
- **Documentation** : rédaction du rapport, capture d'écran des résultats, explication du code et des difficultés rencontrées.

Ce projet nous a permis de relier la théorie à la pratique, tout en développant notre capacité d'analyse, notre autonomie et notre esprit de rigueur, indispensables dans le métier du développement logiciel.

Cahier des charges

2.1 Présentation de l'Institut IPIRNET :

L'institut IPIRNET est un centre de formation professionnelle situé à Berrechid, accrédité par l'État, qui propose plusieurs filières dans les domaines techniques et informatiques, notamment la filière Technicien Spécialisé en Développement Informatique (TSDI). Son objectif principal est de former des jeunes compétents, capables de répondre aux besoins du marché de l'emploi grâce à une formation théorique et pratique.

2.2 Analyse des besoins fonctionnels :

Le projet consiste à développer une application permettant la gestion complète des stagiaires. Les besoins fonctionnels identifiés sont :

- Enregistrement des stagiaires avec numéro d'inscription, nom, prénom, filière, niveau.
- Modification des informations d'un stagiaire.
- Suppression d'un stagiaire.
- Affichage de la liste globale des stagiaires.
- Recherche d'un stagiaire par son numéro d'inscription.
- Affichage par filière.
- Affichage de l'état global des stagiaires.
- Gestion des notes (programmation, TP, logiciel) et génération de bulletins.
- Affichage du résultat (admis, redoublant, lauréat).

2.3 Analyse des besoins techniques :

Pour réaliser cette application, les éléments techniques suivants sont nécessaires :

- **Langage de programmation** : Python (programmation structurée).
- **Système de stockage des données** : Tableaux (arrays).

- **Outils de développement** : éditeur de code Python (IDLE, VS Code...).
- **Méthodologie de développement** : utilisation de procédures et de fonctions, menu interactif, affichage dynamique.

2.4 Contraintes et spécifications :

- L'application doit être exécutable en local sans base de données externe.
- Toutes les opérations doivent vérifier la validité des données (contrôle d'existence, format, etc.).
- L'interface utilisateur doit être simple et basée sur un menu de choix.
- Le programme doit être bien commenté et structuré.
- Le projet doit respecter la date limite de dépôt fixée au **12/05/2025**.

Étude et conception

3.1 Modélisation des données (Tableaux, structures) :

Pour organiser les informations relatives aux stagiaires, nous avons utilisé des **structures de données simples** comme les **tableaux (arrays)**. Chaque tableau représente un ensemble de données du même type :

- n_inscription[] : Numéros d'inscription
- noms[] : Noms et prénoms des stagiaires
- filières[] : Filières
- niveaux[] : Niveaux (1ère Année / 2ème Année)
- note_prog[], note_tp[], note_logiciel[] : Notes des différentes matières

Chaque index dans les tableaux représente un **stagiaire unique**. Ces structures permettent de **gérer dynamiquement** les données et de faciliter les traitements (recherche, modification, suppression...).

3.2 Diagrammes fonctionnels :

La logique du programme repose sur un **menu principal** à partir duquel l'utilisateur peut accéder aux différentes fonctionnalités :

- **Chargement des données**
- **Affichage des stagiaires (globale, par filière, ou individuel)**
- **Ajout, modification, suppression d'un stagiaire**
- **Affichage des notes et du bulletin individuel**
- **Validation des résultats et calcul des moyennes**

Nous avons utilisé des **organigrammes (algorigrammes)** pour représenter le déroulement logique de chaque fonction, ce qui permet une meilleure planification du code.

3.3 Architecture de l'application :

L'application suit une architecture modulaire :

- **Entrée/Sortie** : pour lire et afficher les données
- **Gestion des données** : chargement, modification, suppression
- **Traitement** : calculs des moyennes, validation, tri
- **Interface utilisateur** : menu interactif en mode console

Chaque fonction du programme est bien séparée, ce qui facilite la **lisibilité**, la **modification** et la **réutilisation** du code.

3.4 Choix technologiques :

Le choix s'est porté sur :

- **Langage Python** : pour sa simplicité syntaxique et sa puissance dans la manipulation de données.
- **Programmation structurée** : afin d'organiser le code en procédures claires et distinctes.
- **L'algorithmique** : pour concevoir des solutions logiques avant leur implémentation en Python.

Ces choix ont permis de développer une application **simple, efficace** et **adaptée au contexte éducatif** du centre de formation IPIRNET.

Développement

4.1 Présentation de l'environnement de développement :

Le développement de l'application a été réalisé dans un environnement simple, efficace et adapté aux débutants :

- **Langage utilisé** : Python 3.x
- **Éditeur de code** : Thonny IDE / VS Code (selon préférence)
- **Système d'exploitation** : Windows 10
- **Méthode** : Programmation structurée, basée sur des fonctions claires et indépendantes

L'environnement choisi a permis une prise en main rapide et une exécution fluide des programmes. Il est parfaitement adapté aux projets pédagogiques et à la mise en œuvre d'algorithmes de base.

4.2 Description des modules et fonctions :

Le programme est divisé en plusieurs **modules** ou **fonctions**, chacun ayant un rôle précis :

- `charger_donnees()` : pour initialiser les tableaux des stagiaires
- `afficher_liste_globale()` : pour afficher tous les stagiaires
- `ajouter_stagiaire()` : pour insérer un nouveau stagiaire après vérification
- `rechercher_stagiaire()` : pour localiser un stagiaire par son numéro d'inscription
- `modifier_stagiaire()` : pour modifier le nom, la filière ou le niveau
- `supprimer_stagiaire()` : pour retirer un stagiaire de la liste
- `saisir_notes()` : pour entrer les notes d'un stagiaire
- `afficher_bulletin()` : pour afficher les résultats et les notes d'un stagiaire
- `calculer_moyenne()` : pour calculer la moyenne générale

4.3 Gestion des stagiaires :

(Ajout, modification, suppression, recherche)

La gestion des stagiaires repose sur des **fonctions robustes** permettant :

- **L'ajout** avec contrôle de l'unicité du numéro d'inscription
- **La recherche** rapide d'un stagiaire grâce à son identifiant
- **La modification** de ses informations académiques (nom, filière, niveau)
- **La suppression** à travers un filtrage par numéro d'inscription

Chaque opération affiche un message clair à l'utilisateur confirmant le succès ou l'échec de l'action.

4.4 Gestion des notes :

Les notes sont saisies et stockées sous forme de tableaux :

- **Programmation TH**
- **Programmation TP**
- **Logiciel**

Le programme permet :

- La **saisie sécurisée** des notes (contrôle des entrées)
 - L'**affichage détaillé** du bulletin pour chaque stagiaire
 - Le **calcul de la moyenne pondérée** en fonction des coefficients
-

4.5 Validation et règles métier :

Des règles précises ont été intégrées pour **valider** les résultats :

- **Moyenne générale ≥ 10 → Admis**
- **Moyenne < 10 → Redoublant**
- **Moyenne ≥ 10 avec mention → Lauréat**

Des conditions particulières s'appliquent selon le niveau du stagiaire (1ère ou 2ème année). Les règles métier sont codées sous forme de conditions (if, elif, else) pour garantir une logique claire.

Tests et validation

5.1 Stratégie de test :

Afin d'assurer la **fiabilité** et la **stabilité** de l'application, une stratégie de test progressive a été adoptée, incluant :

- Des **tests unitaires** sur chaque fonction indépendamment
- Des **tests d'intégration** pour vérifier la compatibilité entre les modules
- Des **tests de validation** basés sur des scénarios réels d'utilisation

L'objectif était de s'assurer que chaque opération réponde exactement aux besoins définis dans le cahier des charges.

5.2 Tests unitaires et résultats :

Chaque fonction a été testée séparément :

Fonction testée	Entrée(s)	Résultat attendu	Résultat obtenu	Statut
ajouter_stagiaire()	Données valides	Stagiaire ajouté	✓	Succès
ajouter_stagiaire()	Numéro existant	Refus d'ajout	✓	Succès
rechercher_stagiaire()	Numéro correct	Infos affichées	✓	Succès
modifier_stagiaire()	Données existantes	Données modifiées	✓	Succès
supprimer_stagiaire()	Numéro correct	Stagiaire supprimé	✓	Succès
afficher_bulletin()	Numéro correct	Bulletin affiché	✓	Succès
calculer_moyenne()	Notes valides	Moyenne correcte	✓	Succès

5.3 Tests d'intégration :

Après validation des fonctions individuelles, nous avons effectué des tests d'intégration :

- **Ajout + Affichage** : un stagiaire est ajouté puis affiché dans la liste globale ✓
- **Ajout + Saisie de notes + Bulletin** : un stagiaire reçoit ses notes, son bulletin est généré ✓
- **Suppression + Affichage** : un stagiaire est supprimé et n'apparaît plus dans les listes ✓

Ces tests ont confirmé la **cohérence** entre les modules et le bon enchaînement des opérations dans le menu principal.

5.4 Retours d'utilisation :

Des tests en condition réelle ont été réalisés par d'autres stagiaires de la classe. Leurs retours ont permis d'apporter les améliorations suivantes :

- Amélioration de l'affichage avec des séparateurs (-----)
- Ajout de messages d'erreur plus explicites
- Meilleure gestion des erreurs d'entrée (ex : lettres au lieu de chiffres)

Ces retours ont renforcé la **qualité d'utilisation** du programme et facilité sa prise en main par les utilisateurs non techniques.

Difficultés rencontrées et solutions apportées

Durant la réalisation de ce projet de programmation structurée en Python, plusieurs difficultés ont été rencontrées. Cependant, elles ont été progressivement surmontées grâce à une démarche méthodique, à des recherches personnelles et à l'encadrement du formateur.

6.1 Manipulation des tableaux et structures :

Difficulté :

Au début, la gestion des données à l'aide de tableaux (arrays) était complexe, surtout lorsqu'il fallait manipuler plusieurs informations pour chaque stagiaire (nom, filière, niveau, numéro d'inscription, etc.).

Solution apportée :

Utilisation de tableaux multidimensionnels et structuration des données par catégorie (liste des stagiaires, liste des notes, etc.), avec une bonne organisation des index. Un travail de révision sur les structures conditionnelles et les boucles a permis de mieux les maîtriser.

6.2 Saisie et validation des données :

Difficulté :

Il était difficile de contrôler la validité des entrées utilisateurs (ex : éviter les doublons, vérifier les types des données...).

Solution apportée :

Mise en place de **fonctions de validation** pour chaque saisie (vérification du type, de la longueur, et de l'unicité des numéros d'inscription), et intégration de messages d'erreur clairs pour guider l'utilisateur.

6.3 Gestion de la moyenne générale et des règles de validation :

Difficulté :

Le calcul des moyennes et l'application des règles de validation (Lauréat, Doublant...) nécessitaient une bonne compréhension des coefficients et de la logique métier.

Solution apportée :

Des tests manuels ont été réalisés avec différentes notes pour vérifier les formules. Les règles ont été traduites en conditions simples avec des commentaires explicatifs dans le code Python.

6.4 Interface utilisateur en mode texte :

Difficulté :

Créer un menu clair, lisible et fonctionnel en mode console n'était pas évident au départ, surtout pour structurer les différentes opérations.

Solution apportée :

Un **menu principal avec des sous-menus** a été conçu en utilisant des boucles et des structures conditionnelles. Des séparateurs visuels (=====) ont été ajoutés pour améliorer la lisibilité de l'interface.

6.5 Organisation du projet et respect des consignes :

Difficulté :

La quantité de livrables à fournir (code source, captures d'écran, rapport imprimé, etc.) nécessitait une bonne gestion du temps et de l'organisation.

Solution apportée :

Un **planning de travail** a été mis en place, avec des étapes bien définies : d'abord le développement du code, ensuite les tests, puis la rédaction du rapport. Cela a permis de respecter les délais.

Conclusion et perspectives

Ce projet de développement d'une application de gestion des stagiaires du centre IPIRNET, réalisé en Python et basé sur les principes de l'algorithmique, nous a permis de mettre en pratique les compétences acquises au cours de notre formation. En passant par les étapes de conception, de modélisation, de codage et de tests, nous avons pu renforcer notre compréhension de la programmation structurée et développer notre autonomie dans la résolution de problèmes concrets.

Nous avons également appris à respecter un cahier des charges précis, à structurer notre travail de manière rigoureuse, et à produire un livrable fonctionnel répondant aux besoins exprimés. Ce projet a renforcé notre esprit logique, notre capacité d'organisation ainsi que notre sens de la responsabilité.

Perspectives :

Dans un futur proche, ce projet pourrait être enrichi par l'ajout d'une interface graphique (GUI), l'intégration d'une base de données pour un meilleur stockage des informations, ou encore le déploiement de l'application en réseau afin de permettre un usage multi-utilisateurs. Ces évolutions permettraient d'optimiser l'efficacité de l'outil et de mieux répondre aux exigences d'un centre de formation moderne.

Annexes

8.1 Codes sources :

Dans cette section, nous présentons le code source de l'application. Il est organisé par modules/fonctions avec des commentaires clairs pour faciliter la lecture et la compréhension. Chaque fonction est précédée d'une brève description de son rôle et de son utilité dans le projet.

➤ Déclaration des tableaux :

```
15     stagiaires_globale = [  
16         Stagiaire(1001, "Ahmed Zaki", "TSDI", "1ère année"),  
17         Stagiaire(1002, "Sofia Benali", "TSGE", "2ème année"),  
18         Stagiaire(1003, "Yassine Ait", "TGI", "1ère année"),  
19         Stagiaire(1004, "Meryem Lahlou", "OPAD", "2ème année"),  
20         Stagiaire(1005, "Khalid Hani", "TSDI", "1ère année"),  
21         Stagiaire(1006, "Laila Fassi", "TGI", "2ème année"),  
22         Stagiaire(1007, "Omar Tazi", "OPAD", "1ère année"),  
23         Stagiaire(1008, "Rania Lahrach", "TSGE", "1ère année")  
24     ]
```

➤ Fonctions de gestion des stagiaires :

```
11 def ajouter_stagiaire(num_inscription, nom, prenom, niveau, filiere):  
12     # Vérification si le stagiaire existe déjà  
13     for stagiaire in stagiaires:  
14         if stagiaire['num_inscription'] == num_inscription:  
15             print(f"Le stagiaire avec le numéro {num_inscription} existe déjà.")  
16             return  
17     # Ajout du stagiaire  
18     stagiaires.append({  
19         'num_inscription': num_inscription,  
20         'nom': nom,  
21         'prenom': prenom,  
22         'niveau': niveau,  
23         'filiere': filiere  
24     })  
25     afficher_stagiaires()
```

```
# Fonction pour modifier un stagiaire
def modifier_stagiaire(num_inscription, nom=None, prenom=None, niveau=None, filiere=None):
    for stagiaire in stagiaires:
        if stagiaire['num_inscription'] == num_inscription:
            if nom:
                stagiaire['nom'] = nom
            if prenom:
                stagiaire['prenom'] = prenom
            if niveau:
                stagiaire['niveau'] = niveau
            if filiere:
                stagiaire['filiere'] = filiere
            print(f"\nStagiaire {num_inscription} modifié:")
            afficher_stagiaires()
            return
    print(f"Stagiaire avec le numéro {num_inscription} non trouvé.")
```

```
# Fonction pour supprimer un stagiaire
def supprimer_stagiaire(num_inscription):
    global stagiaires
    stagiaires = [stagiaire for stagiaire in stagiaires if stagiaire['num_inscription'] != num_inscription]
    print(f"\nStagiaire avec le numéro {num_inscription} supprimé.")
    afficher_stagiaires()
```

➤ Fonctions de gestion des notes :

```
zineb.py  Programme p.y.py  gestion_notes p.y.py X
C: > Users > acer > OneDrive > Bureau > gestion_notes p.y.py > Stagiaire > __init__
1  class Stagiaire:
2      def __init__(self, num_inscription, nom, prenom, filiere, niveau, note_prog_th, note_prog_tp, note_logiciel):
3          self.num_inscription = num_inscription
4          self.nom = nom
5          self.prenom = prenom
6          self.filiere = filiere
7          self.niveau = niveau
8          self.note_prog_th = note_prog_th
9          self.note_prog_tp = note_prog_tp
10         self.note_logiciel = note_logiciel
11
12         # Coefficients
13         self.coef_prog_th = 5
14         self.coef_prog_tp = 5
15         self.coef_logiciel = 3
16
17         # Calcul de la moyenne
18         self.moyenne_generale = self.calcul_moyenne_generale()
19
20     def calcul_moyenne_generale(self):
21         total_notes = (
22             self.note_prog_th * self.coef_prog_th +
23             self.note_prog_tp * self.coef_prog_tp +
24             self.note_logiciel * self.coef_logiciel
25         )
26         total_coefs = self.coef_prog_th + self.coef_prog_tp + self.coef_logiciel
27         return total_notes / total_coefs
28
```

```

def afficher_ligne_bulletin(self, module, note, coef):
    note_coef = note * coef
    print(f"| {module:<20} | {note:<7} | {coef:<4} | {note_coef:<10} | {'...':<11} |")

def afficher_bulletin(self):
    total = (
        self.note_prog_th * self.coef_prog_th +
        self.note_prog_tp * self.coef_prog_tp +
        self.note_logiciel * self.coef_logiciel
    )
    moyenne = self.moyenne_generale
    resultat = "Réussi" if moyenne >= 10 else "Échoué"

    print("\nGroupe IPIRNET")
    print("    Berrechid")
    print("                BULLETIN DE NOTES")
    print(f"Nom et prénom : {self.nom} {self.prenom:<25} Filière : {self.filiere}")
    print(f"Niveau : {self.niveau:<39} Année : 2022/2023")
    print("-----|-----|-----|-----|-----|")
    print("| Module          | Note    | Coef    | Note*Coef | Observation |")
    print("-----|-----|-----|-----|-----|")

    self.afficher_ligne_bulletin("Programmation TH", self.note_prog_th, self.coef_prog_th)
    self.afficher_ligne_bulletin("Programmation TP", self.note_prog_tp, self.coef_prog_tp)
    self.afficher_ligne_bulletin("Logiciel", self.note_logiciel, self.coef_logiciel)

    print("-----|-----|-----|-----|-----|")

```

```

# Espaces dans la colonne Module
print(f"| {'':<22} | Total    : {total:<8} |")
print(f"| {'':<22} | Moyenne : {moyenne:<8.2f} |")
print(f"| {'':<22} | Résultat: {resultat:<8} |")
print("=" * 66)

# === Exemple de stagiaire ===

stagiaire = Stagiaire(1002, "Rachid", "El Amrani", "Génie civil", "2ème année", 9.5, 10.0, 11.5)
stagiaire.afficher_bulletin()

```

➤ Menu principal :

```

# F. Menu
def menu():
    print("\n=== Menu de gestion de l'école IPIRNET ===")
    print("1. Afficher l'état global des stagiaires")
    print("2. Afficher l'état des stagiaires trié par numéro d'inscription")
    print("3. Afficher l'état des stagiaires trié par nom et prénom")
    print("4. Afficher les stagiaires par filière")
    print("5. Quitter")
    return input("\nEntrez votre choix (1-5) : ")

```

```

# G. Gérer le menu
def gerer_menu():
    stagiaires_globale = charger_donnees()

    while True:
        choix = menu()

        if choix == "1":
            print("\n=== État global des stagiaires ===")
            afficher_etat_global(stagiaires_globale)
        elif choix == "2":
            print("\n=== État des stagiaires trié par numéro d'inscription ===")
            stagiaires_tries_num = trier_par_num_inscription(stagiaires_globale)
            afficher_etat_global(stagiaires_tries_num)
        elif choix == "3":
            print("\n=== État des stagiaires trié par nom et prénom ===")
            stagiaires_tries_nom = trier_par_nom_prenom(stagiaires_globale)
            afficher_etat_global(stagiaires_tries_nom)
        elif choix == "4":
            print("\n=== Affichage des stagiaires par filière ===")
            afficher_par_filiere(stagiaires_globale)
        elif choix == "5":
            print("\nMerci d'avoir utilisé l'application. À bientôt !")
            break
        else:
            print("\nChoix invalide. Veuillez entrer un numéro entre 1 et 5.")

```

```

# H. Exécution
if __name__ == "__main__":
    gerer_menu()

```

8.2 CAPTURES D'ÉCRAN DES MENUS :

➤ Menu principal :

```
PS C:\Users\acer> & C:/Users/acer/AppData/Local/Programs/Python/Python313/python.exe c:/Users/acer/OneDrive/Bureau/zineb.py

=== Menu de gestion de l'école IPIRNET ===
1. Afficher l'état global des stagiaires
2. Afficher l'état des stagiaires trié par numéro d'inscription
3. Afficher l'état des stagiaires trié par nom et prénom
4. Afficher les stagiaires par filière
5. Quitter

Entrez votre choix (1-5) : 0
0 ^ 0                                     Ln 94, Col 43   Spaces: 4   UTF-8   CRLF
```

• Description :

Le menu principal permet à l'utilisateur d'accéder à toutes les fonctionnalités du système. Il est affiché en boucle tant que l'utilisateur ne choisit pas l'option « Quitter ».

➤ Charger l'état globale des stagiaires :

```
Entrez votre choix (1-5) : 1

=== État global des stagiaires ===
Groupe IPIRNET
Berrechid
LISTE DES STAGIAIRES GLOBALE
Année : 2022/2023
-----
+-----+-----+-----+-----+
| Num inscription | Nom Prenom | Filière | Niveau |
+-----+-----+-----+-----+
| 1001            | Ahmed Zaki | TSDI    | 1ère année |
| 1002            | Sofia Benali | TSGE    | 2ème année |
| 1003            | Yassine Ait | TGI     | 1ère année |
| 1004            | Meryem Lahlou | OPAD    | 2ème année |
| 1005            | Khalid Hani | TSDI    | 1ère année |
| 1006            | Laila Fassi | TGI     | 2ème année |
| 1007            | Omar Tazi | OPAD    | 1ère année |
| 1008            | Rania Lahrach | TSGE    | 1ère année |
+-----+-----+-----+-----+
```

➤ L'état des stagiaires triés par numéro d'inscription :

Entrez votre choix (1-5) : 2

=== État des stagiaires trié par numéro d'inscription ===

Groupe IPIRNET

Berrechid

LISTE DES STAGIAIRES GLOBALE

Année : 2022/2023

Num inscription	Nom Prenom	Filière	Niveau
1001	Ahmed Zaki	TSDI	1ère année
1002	Sofia Benali	TSGE	2ème année
1003	Yassine Ait	TGI	1ère année
1004	Meryem Lahlou	OPAD	2ème année
1005	Khalid Hani	TSDI	1ère année
1006	Laila Fassi	TGI	2ème année
1007	Omar Tazi	OPAD	1ère année
1008	Rania Lahrach	TSGE	1ère année

➤ L'état des stagiaires triés par nom & prénom :

Entrez votre choix (1-5) : 3

=== État des stagiaires trié par nom et prénom ===

Groupe IPIRNET

Berrechid

LISTE DES STAGIAIRES GLOBALE

Année : 2022/2023

Num inscription	Nom Prenom	Filière	Niveau
1001	Ahmed Zaki	TSDI	1ère année
1005	Khalid Hani	TSDI	1ère année
1006	Laila Fassi	TGI	2ème année
1004	Meryem Lahlou	OPAD	2ème année
1007	Omar Tazi	OPAD	1ère année
1008	Rania Lahrach	TSGE	1ère année
1002	Sofia Benali	TSGE	2ème année
1003	Yassine Ait	TGI	1ère année

➤ L'affichage des stagiaires par filière :

Entrez votre choix (1-5) : 4

=== Affichage des stagiaires par filière ===

Groupe IPIRNET

Berrechid

LISTE DES STAGIAIRES GLOBALE

Année : 2022/2023

Filière : TSDI

Num inscription	Nom Prenom	Niveau
1001	Ahmed Zaki	1ère année
1005	Khalid Hani	1ère année

Groupe IPIRNET

Berrechid

LISTE DES STAGIAIRES GLOBALE

Année : 2022/2023

Groupe IPIRNET

Berrechid

LISTE DES STAGIAIRES GLOBALE

Année : 2022/2023

Filière : TSGE

Num inscription	Nom Prenom	Niveau
1002	Sofia Benali	2ème année
1008	Rania Lahrach	1ère année

Groupe IPIRNET

Berrechid

LISTE DES STAGIAIRES GLOBALE

Année : 2022/2023

Filière : TGI

Num inscription	Nom Prenom	Niveau
1003	Yassine Ait	1ère année
1006	Laila Fassi	2ème année


```
Groupe IPIRNET
Berrechid
LISTE DES STAGIAIRES GLOBALE
Année : 2022/2023
```

```
Filière : OPAD
```

```
-----
+-----+-----+-----+-----+
| Num inscription | Nom Prenom | Niveau | |
+-----+-----+-----+-----+
| 1004            | Meryem Lahlou | 2ème année | |
| 1007            | Omar Tazi    | 1ère année | |
+-----+-----+-----+-----+
```

➤ Charger du bulletin de notes :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: gestion_
PS C:\Users\acer> & C:/Users/acer/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/acer/OneDrive/Bureau/gestion_notes p.y.py

Groupe IPIRNET
Berrechid
BULLETIN DE NOTES
Nom et prénom : Rachid El Amrani Filière : Génie civil
Niveau : 2ème année Année : 2022/2023
-----
| Module | Note | Coef | Note*Coef | Observation |
-----
| Programmation TH | 9.5 | 5 | 47.5 | ...
| Programmation TP | 10.0 | 5 | 50.0 | ...
| Logiciel | 11.5 | 3 | 34.5 | ...
-----
| Total : 132.0 |
| Moyenne : 10.15 |
| Résultat: Réussi |
=====
PS C:\Users\acer>
```

➤ Quitter le programme :

```
Entrez votre choix (1-5) : 5
```

```
Merci d'avoir utilisé l'application. À bientôt !
```

```
PS C:\Users\acer> █
```

8.3 DOCUMENTATION COMPLÉMENTAIRE

➤ Algorithme général de l'application

Début

Définir la structure Stagiaire avec :

- Numéro d'inscription
- Nom et prénom
- Filière
- Niveau

Procédure charger_donnees()

Créer une liste de stagiaires avec des données
prédéfinies

Retourner cette liste

Procédure trier_par_num_inscription(liste)

Trier la liste selon le numéro d'inscription croissant
Retourner la liste triée

Procédure trier_par_nom_prenom(liste)

Trier la liste selon l'ordre alphabétique des noms
Retourner la liste triée

Procédure afficher_etat_global(liste)

Afficher l'en-tête du tableau

Pour chaque stagiaire dans la liste faire

Afficher son numéro, nom, filière et niveau

Fin Pour

Procédure afficher_par_filiere(liste)

Pour chaque filière unique dans la liste faire

Afficher en-tête + tableau

Pour chaque stagiaire de cette filière faire

Afficher ses infos

Fin Pour

Fin Pour

Procédure menu()

Afficher les options :

1. État global
2. Tri par numéro

```

        3. Tri par nom
        4. Par filière
        5. Quitter
Lire choix
Retourner choix

Procédure gerer_menu()
    Appeler charger_donnees() et stocker dans
stagiaires_globale
    Répéter
        Appeler menu() → choix
        Selon choix faire
            Cas "1" :
afficher_etat_global(stagiaires_globale)
            Cas "2" : trier_par_num_inscription + afficher
            Cas "3" : trier_par_nom_prenom + afficher
            Cas "4" : afficher_par_filiere
            Cas "5" : afficher "Merci" et quitter la
boucle
            Sinon : afficher "Choix invalide"
        Fin Selon
    Jusqu'à ce que choix = "5"

    Appeler gerer_menu()

Fin

```